HR & Payroll — Lean Rebuild Plan (Plan B)

Project: Easy School Management System **Supersedes:** `HR-Payroll-Module-Plan.md` (Plan A — the Dropware-shaped port) **Date:** 2026-08-25

---

1. The finding that changes everything

I checked what the existing HR code actually does. It does nothing:

File	Reality
`Repositories/Implementations/HR/EmployeeRepository.cs`	233 lines, **202 commented out** — zero methods, empty class
`Repositories/Interfaces/HR/IEmployeeRepository.cs`	**Empty interface**
`Controllers/EmployeeController.cs`	**Zero endpoints** — only a constructor
`Services/Implementations/HR/EmployeeService.cs`	132 lines, **109 commented out**
`Controllers/SalaryController.cs`	**Zero endpoints**
`Services/Implementations/HR/SalaryService.cs`	67 lines, **41 commented out**

There is no working HR module to preserve. Employee and Salary are dead scaffolding — not reachable from the API at all. Only Designation, HRGrade and Teacher are live.

This is good news. It means: **no migration burden, no compatibility constraints, green field.** Plan A was written as if we were extending a working module. We aren't. So Plan B deletes the scaffolding and builds small and correct instead of large and ported.

---

2. Why Plan A was too big

Dropware's HRM is a **corporate ERP** — 13 setup forms, 39 transaction forms, 9 reports, built for thousands of employees with EOBI, tax slabs, medical claims, loan disbursement queues, provident fund and multi-company scoping.

Sir Syed Model School has on the order of **100–200 staff**. Sizing reality:

```
200 staff x 31 days      =   6,200 attendance rows / month
                         =    ~74,000 rows / year
Payroll run              =     200 payslips / month
```

SQL Server does not notice this. **Data volume is a non-issue.** Every complexity in Dropware that exists to handle scale is dead weight here. What actually matters is that the school clerk can enter a month of attendance quickly and that payroll is correct.

So Plan B optimises for **fewer moving parts**, not for throughput.

---

3. Five design decisions

D1. Compute on save — no "posting" runs

Dropware needs three manual steps: enter attendance → run Attendance Posting → run Salary Posting. Late/early/worked minutes is *arithmetic on two times against a shift*. Do it in the service when the row is saved.

Kills: the Attendance Posting screen, the `AttClosingDay` intermediate table, and one stored procedure. **Also kills** the single biggest support-call category in systems like this — "I entered attendance but the salary is wrong" (because someone forgot to re-post).

D2. Rules in C#, reports in SQL

Dropware buries payroll logic in `SPHRAttClosingMonth`. Stored procedures can't be unit-tested, code-reviewed, or diffed properly in git.

- **Payroll calculation → C#** (`PayrollCalculator`, pure function, no DB access)
- **Reports → raw SQL / SPs** (set-based, already the house pattern in `School.API/Scripts/`)

The repo currently has **zero test projects**. Add one — `School.Tests` — covering `PayrollCalculator` only. That single class is where money is computed; everything else is CRUD.

D3. JSON columns instead of child tables

EF Core 8.0.8 (already referenced) supports `OwnsMany(...).ToJson()`. Use it for data that is always loaded as a whole and never queried across rows:

- Rule slabs → `SlabsJson` on the component (kills a table + a child grid form)
- Salary structure lines → `ComponentsJson` (kills a table)
- Payslip breakdown → `BreakdownJson` (kills a table)

D4. One flag beats one module

Late exemption in Dropware is a table, a setup form, a lookup SP and a join. Here it is a boolean `IgnoreLate` on the attendance row, toggled from the attendance grid. **Kills a table and a form.**

D5. Aggregate, don't paginate punches

Hikvision export has many punches per person per day. Aggregate to first-in / last-out **at import time** and write straight to `HrAttendance`. Keep the uploaded file + a batch id for audit/reversal.

Kills the raw punch table entirely. (This also replaces Plan A section 4.2's shared raw table — a shared *import service* with two writers is cleaner than a shared table with a `PersonType` discriminator.)

---

4. Schema — 9 tables (Plan A had ~30)

```
HrShift            TimeIn, TimeOut, GraceMinutes, WorkHours, HalfDayMinutes, OffDay1, OffDay2
HrHoliday          Date, Name, Type
HrLeaveType        Name, IsPaid, DeductDays, MaxPerYear          (seeded, no CRUD screen)
HrAttendance   *   EmployeeId, AttDate, TimeIn, TimeOut, ShiftId, Status,
                   LateMinutes, EarlyMinutes, WorkedMinutes, LeaveTypeId,
                   IgnoreLate, Source, Remarks          <- computed on save
HrLeave            EmployeeId, LeaveTypeId, FromDate, ToDate, Days, Status, Reason
HrPayComponent *   the whole rule engine, one table      <- see section 5
HrEmployeePay      EmployeeId, EffectiveFrom, Basic, ComponentsJson, IsActive
HrPayrollRun       Year, Month, Status (Draft|Locked|Paid), LockedBy, LockedAt
HrPayslip          RunId, EmployeeId, Gross, Deductions, Net, BreakdownJson,
                   PaidDate, PaymentMethod, Reference

(+2 later, Phase 4 only)   HrLoan, HrLoanInstallment
```

`Employee` gets four new columns and nothing else: `ShiftId`, `MachineCardNo`, `OffDay1`, `OffDay2`. Its existing flat `BasicSalary` / `Allowances` / `Deductions` fields are dropped in favour of `HrEmployeePay` (they're unused today — nothing reads them).

Indexes (all three matter):

```
UNIQUE (EmployeeId, AttDate)                                -- HrAttendance, prevents double entry
INDEX  (AttDate) INCLUDE (EmployeeId, Status, LateMinutes)  -- HrAttendance, drives every report
UNIQUE (RunId, EmployeeId)                                  -- HrPayslip, prevents double payslip
```

---

5. The rule engine — one table, still fully generic

Requirement stands: Sir Syed deducts **per minute late**; other schools differ; formula must be data. Plan A used 3 tables and ~20 columns. Same capability in **one** table:

Column	Values
`Kind`	`Earning` / `Deduction`
`CalcType`	`Fixed`, `PercentOf`, `PerLateMinute`, `PerAbsentDay`, `PerOvertimeHour`, `Slab`, `Formula`
`Rate`	the amount, percent, or per-unit rate
`RateSource`	`Fixed` (typed in) / `Derived` (computed from salary)
`BaseOn`	`Basic` / `Gross` — base for `PercentOf` and `Derived`
`DivisorDays`	`30` / `Calendar` / `WorkingDays`
`DivisorHours`	null = take from the shift
`GraceMinutes`, `FreeUnitsPerMonth`	forgiveness
`CapPerDay`, `CapPerMonth`	limits
`SlabsJson`	slab bands, JSON
`Formula`	NCalc escape hatch
`ScopeType` + `ScopeId`, `Priority`	`Global` / `Grade` / `Designation` / `Employee`, most-specific wins
`EffectiveFrom`, `EffectiveTo`	history stays reproducible

Sir Syed = one row:

```
Kind=Deduction  CalcType=PerLateMinute  RateSource=Derived  BaseOn=Basic
DivisorDays=30  DivisorHours=null(from shift)  GraceMinutes=10  CapPerDay=500
```

```
rate      = Basic / 30 / ShiftHours / 60
deduction = MIN((LateMinutes - 10) * rate, 500)
```

Flat Rs 20/min school → `RateSource=Fixed, Rate=20`. Half-day-after-3-lates school → `CalcType=Slab`, `FreeUnitsPerMonth=3`. **No code change in any case.**

---

6. Screens — 10 (Plan A had 43)

#	Screen	Replaces in Plan A
1	**Payroll Setup** — one page, 4 tabs: Shifts · Holidays · Leave Types · Pay Components (with a live "25 min late → Rs X" preview)	S4, S6, S7, S9, S10, S11, S12, S13
2	**Employee** (rebuild) — existing tabs + a Payroll tab (shift, card no, salary structure)	T1, T2, T15
3	**Attendance** — one month grid, employee × day; click a cell to set time/status; bulk fill; `IgnoreLate` toggle	T3, T4, T5, T6
4	**Leave** — one list, apply + approve via status actions	T7, T8
5	**Payroll Run** — generate → review grid → adjust → lock → mark paid, one page	T9, T12, T13, T14, S14
6	**Payslip** — print PDF + send WhatsApp	R7
7	**Salary Sheet** — grid + Excel export (bank advice = an export option here)	R6, R8
8	**Attendance Report** — one page, report-type dropdown (daily / monthly / late / absent)	R1, R2, R3, R4

#	Screen	Replaces in Plan A
9	**Loans & Advances** (Phase 4)	T10, T11, R9
10	**Attendance Import** (Phase 4)	Hikvision

Everything else in Plan A — departments, employee shift assignment, leave entitlement ledgers, EOBI/PF, tax slabs, final settlement, card printing, bank setup, month-days setup — is **cut**. See section 9.

---

7. Performance plan

The bottleneck is round trips, not rows. Concrete targets:

Payroll run — exactly 4 queries, one transaction:

```
1. employees + active pay structures   (1 query, AsNoTracking, projected)
2. attendance aggregate for the month  (1 query, GROUP BY EmployeeId)
3. approved leave aggregate            (1 query, GROUP BY EmployeeId)
4. due loan installments               (1 query)
   -> PayrollCalculator in memory (pure C#, no DB)
   -> AddRange(payslips) + one SaveChanges inside one transaction
```

Target: **under 2 seconds for 200 employees.** The failure mode to avoid is a `foreach (employee)` loop that queries per person — that turns 4 queries into 800.

Other measures:

- `AsNoTracking()` on every read. The Academic and Fee repos already do this consistently; the HR ones don't — rebuild them to match.
- `IMemoryCache` for shifts, holidays and pay components (they change monthly at most). `AddMemoryCache()` is **already registered** in `Program.cs:44` and unused for this.
- Attendance month grid: **one** projected query, no `Include()`. 200 × 31 cells is fine to send at once; no paging needed at this size.
- Add **response compression** to `Program.cs` — currently not configured. The attendance grid and salary sheet are the largest JSON payloads in the app and compress ~80%.
- Add `EnableRetryOnFailure()` to the SQL Server connection — also currently not configured.
- Reports as raw SQL only. Never materialise a report through EF entities.

---

8. Phases — 12–16 days (Plan A was 24–34)

Phase 1 — Foundation + Attendance (5–6 days) — *usable on its own*

- [ ] Delete the dead HR scaffolding (Employee/Salary controller, service, repo, interfaces)
- [ ] `HrShift`, `HrHoliday`, `HrLeaveType`, `HrAttendance` + indexes; one idempotent `Scripts/HrPayroll.sql`
- [ ] Employee rebuild: working CRUD + the 4 new columns
- [ ] Screen 1 (Setup: Shifts + Holidays tabs), Screen 2 (Employee), Screen 3 (Attendance grid)
- [ ] `AttendanceCalculator` — late/early/worked on save
- [ ] Screen 8 (Attendance Report)

Ships: the school runs staff attendance end-to-end. No payroll yet.

Phase 2 — Payroll engine (4–5 days)

- [ ] `HrPayComponent`, `HrEmployeePay`, `HrPayrollRun`, `HrPayslip`
- [ ] `School.Tests` project + `PayrollCalculator` unit tests (late deduction, grace, caps, slabs, prorata)

- [ ] Setup screen: Pay Components tab with live preview
- [ ] Screen 5 (Payroll Run), Screen 6 (Payslip PDF), Screen 7 (Salary Sheet + Excel)
- [ ] The 4-query run + response compression + EF retry

Ships: full monthly payroll for Sir Syed's per-minute rule.

Phase 3 — Leave (2 days)

- [ ] `HrLeave` + Screen 4; feed approved leave into attendance status

Phase 4 — Optional extras (3–4 days, only if asked)

- [ ] Loans & advances (Screen 9)
- [ ] Hikvision import (Screen 10) — shared `AttendanceImportService`, aggregate at import, two writers: staff → `HrAttendance`, students → existing `StdAttendence`
- [ ] Payslip over WhatsApp via the existing WAHA sender

---

9. What I cut, and why

Cut	Why it's safe
Department table	`Designation` already groups staff. Add later if the client asks — it's a lookup + FK.
Employee Shift Assignment screen	One `ShiftId` on Employee covers a school. Per-day rosters are a factory problem.
Leave entitlement ledger	Schools track leave informally. Ship `HrLeave` first; add balances only if requested.
Attendance Posting screen	D1 — computed on save.
Raw punch table	D5 — aggregated at import.
Late Exemption table + form	D4 — a flag on the attendance row.
Rule slab child table + form	D3 — JSON.
Salary detail + structure line tables	D3 — JSON.
EOBI / PF / tax slabs / advance tax	Not applicable to a private school payroll. Add a fixed-amount `HrPayComponent` if any is ever needed.
Final settlement, card print, employee hierarchy, medical claims	Genuine ERP features with no school use case.
Salary Month Lock as a table+screen	It's a `Status` column on `HrPayrollRun`.
Bank setup	A text field on the employee is enough for a bank advice export.

Everything cut is **additive later** — none of it requires reshaping the 9 core tables.

---

10. Decisions needed before Phase 1

1. **Teachers** — `SMSTeacher` and `Employees` are separate tables. Do teachers draw salary through this module? If yes, merge teachers into `Employees` **now**; retrofitting later is painful. *(This is the one open question that can't be deferred.)*
2. Grace minutes, daily cap, and month-days basis (30 vs calendar) for Sir Syed.
3. Weekly off — Sunday only, or Friday half-day + Sunday?
4. Do summer/winter timings differ? (If yes, `HrShift` gets a date range and employees get a shift *per period* — a small change, but cheaper to know now.)

Questions 2–4 have safe defaults (10 min grace, no cap, 30 days, Sunday off, single shift) and can be changed from the Setup screen afterwards without a migration. **Only question 1 blocks.**
